

Introduction to C Programming

Unit 2: Comprehensive Lecture Notes, Structure, Compilation Pipeline, Tokens, Operators, and Memory Concepts



Real Examples &
Code



Compilation
Diagrams



Exam Point-Wise Notes

Introduction & History of C

- ▶ **Origin:** Developed by **Dennis Ritchie** in 1972 at AT&T's Bell Laboratories (USA) to construct the UNIX Operating System.
- ▶ **Predecessor:** Evolved from B language (developed by Ken Thompson) and BCPL language.
- ▶ **Middle-Level Language:** Combines low-level assembly language power (direct hardware control) with high-level language user abstractions.
- ▶ **Procedural & Structured:** Focuses on functions, modules, and step-by-step procedure execution.
- ▶ **Portability:** Code written on one machine can execute on different hardware architectures with minimal modification.
- ▶ **Fast Execution Speed:** Compiles directly to machine assembly instructions with minimal runtime overhead.
- ▶ **Real-Life Analogy:** C is like the *engine and chassis* of modern software; operating systems, Python interpreters, and database kernels (MySQL/SQLite) are built using C.



Structure of a C Program

The 6 Standard Sections

- ▶ **1. Documentation Section:** Comments (`/* ... */`) specifying author, program purpose, and date.
- ▶ **2. Link Section:** Includes preprocessor header files (e.g., `#include`) providing library functions.
- ▶ **3. Definition Section:** Defines symbolic constants and macros using `#define`.
- ▶ **4. Global Declaration:** Declares global variables and function prototypes accessible anywhere.
- ▶ **5. `main()` Function:** Mandatory entry point of every C program. Execution starts and ends here.
- ▶ **6. Sub-program Section:** Contains user-defined function definitions called within `main()`.

Standard Annotated Code Structure

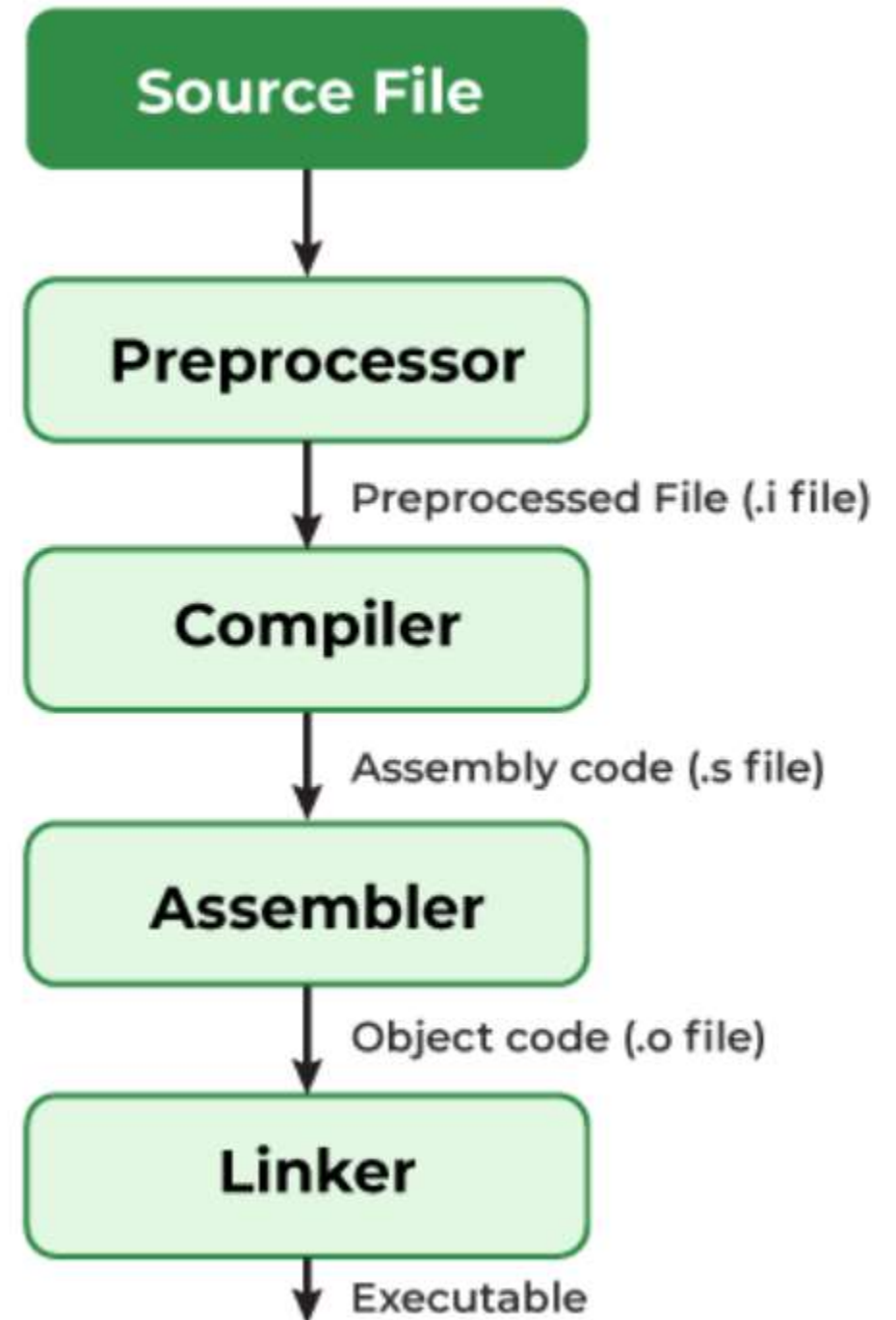
```
/* 1. Documentation: Bank Account Interest */
#include // 2. Link Section
#define RATE 0.05 // 3. Definition Section

float balance = 1000.0; // 4. Global Declaration

// 5. Main Function Execution Block
int main() {
    float interest = balance * RATE; // Local var
    printf("Interest = %.2f\n", interest);
    return 0; // Signals success status
}
```

Compilation & Linking Pipeline

- ▶ **1. Preprocessing** (`.c` → `.i`): Preprocessor removes comments, expands `#include` header files, and substitutes `#define` macros.
- ▶ **2. Compilation** (`.i` → `.s`): Compiler translates expanded source code into assembly instructions and checks syntax errors.
- ▶ **3. Assembly** (`.s` → `.o` / `.obj`): Assembler converts assembly code into binary machine object code.
- ▶ **4. Linking** (`.o` → `.exe`): Linker merges object code with system library code (e.g. `printf`) into an executable file.
- ▶ **Real-Life Analogy:** Like baking: recipe prep (Preprocessing), steps translation (Compiler), mixing ingredients (Assembler), and garnishing/packaging (Linker).



C Tokens: Core Building Blocks

A **Token** is the smallest individual unit in a C program that is meaningful to the compiler.



Keywords

32 predefined reserved words with fixed meanings (`int`, `return`, `if`, `float`, `while`). Must be in lowercase.



Identifiers

User-defined names given to variables, functions, and arrays. Must begin with a letter or underscore `\_` (e.g., `total\_score`).



Literals & Symbols

Fixed constant values (`10`, `3.14`, `A`) and special symbols (`{}`, `()`, `;`, `,`) that structure the syntax.

Real-Life Analogy: Tokens are like words, punctuation, and grammar rules in human languages—without them, sentences (statements) cannot be formed.

Variables & Constants in C

Variables

- ▶ A named memory location used to store data that can change during program execution.
- ▶ Must be declared before use with a data type: `int speed = 60;`
- ▶ **Naming Rules:** Alphabets, digits, and underscores allowed; first character cannot be a digit; case-sensitive; no keywords allowed.
- ▶ **Real-Life Analogy:** A reusable whiteboard inside a locker—you can erase and write new values anytime.

Constants

- ▶ Fixed values that cannot be modified by the program once defined.
- ▶ **Defined using `const`:** `const float PI = 3.14159;` (Type-safe constant).
- ▶ **Defined using `#define`:** `#define MAX_LIMIT 100` (Macro preprocessor replacement).
- ▶ **Real-Life Analogy:** An engraved metallic plaque—once stamped, the value cannot be modified.

C Data Types & Memory Layout

Category	Data Type	Size (32-bit)	Value Range	Real-World Example
P PRIMARY	int	2 or 4 Bytes	-32,768 to 32,767 (2B)	Student Roll Number, Age (e.g., 21)
P PRIMARY	float	4 Bytes	1.2E-38 to 3.4E+38 (6 decimals)	Account Balance, Temperature (e.g., 98.6)
P PRIMARY	double	8 Bytes	2.3E-308 to 1.7E+308 (15 decimals)	GPS Coordinates, Scientific Constants
P PRIMARY	char	1 Byte	-128 to 127 or ASCII 'A'-'Z'	Letter Grades ('A', 'B'), Keypresses
D DERIVED	Array, Pointer	Varies	Contiguous memory blocks	List of 60 student marks
U USER-DEFINED	struct, enum	Sum of members	Custom composite fields	Employee record (ID, Name, Salary)

Note: void specifies an empty data type with no return value (e.g., void display()).

Operators in C Programming

Arithmetic & Assignment

- ▶ **Arithmetic:** `+`, `-`, `*`, `/`, `%` (Modulus returns remainder: `7 % 3 = 1`).
- ▶ **Shorthand Assignment:** `+=`, `-=`, `*=`, `/=` (`x += 5` same as `x = x + 5`).
- ▶ **Increment/Decrement:** `++a` (pre-increment) vs `a++` (post-increment).

Relational & Logical

- ▶ **Relational:** `==`, `!=`, `>`, `<`, `>=`, `<=` (Returns 1 for true, 0 for false).
- ▶ **Logical AND (`&&`):** True if both conditions are true.
- ▶ **Logical OR (`||`):** True if at least one condition is true.
- ▶ **Logical NOT (`!`):** Inverts truth state.

Bitwise & Special

- ▶ **Bitwise:** `&` (AND), `|` (OR), `^` (XOR), `~` (NOT), `<<` (Left Shift), `>>` (Right Shift).
- ▶ **sizeof():** Returns memory size in bytes.
- ▶ **Ternary Operator (`?:`):** `condition ? expr1 : expr2;` (Inline `if-else`).

Operator Precedence & Evaluation

Precedence & Associativity Rules

- ▶ **Precedence:** Determines which operator executes first in a multi-operator expression.
- ▶ **Associativity:** Defines execution direction (Left-to-Right or Right-to-Left) when operators have equal precedence.
- ▶ Parentheses `()` have the highest priority and override default execution order.
- ▶ Unary & Assignment operators evaluate from **Right to Left**.
- ▶ Arithmetic operators evaluate before Relational, Logical, and Assignment operators.

Step-by-Step Expression Evaluation

```
int a = 10, b = 5, c = 2, result;  
  
// Expression: result = a + b * c / (b - 3);  
  
// Step 1: Parentheses (b - 3) = 2  
// Expr: a + b * c / 2  
  
// Step 2: Multiplication b * c = 10  
// Expr: a + 10 / 2  
  
// Step 3: Division 10 / 2 = 5  
// Expr: 10 + 5  
  
// Step 4: Addition 10 + 5 = 15  
result = 15;
```

Type Conversion & Type Casting

Implicit Conversion (Type Promotion)

- ▶ Performed **automatically** by the C compiler during arithmetic operations.
- ▶ Promotes lower data types to higher data types to prevent precision loss.
- ▶ **Hierarchy:** ``char`` → ``int`` → ``float`` → ``double``.

```
int count = 5;  
float price = 2.5;  
float total = count * price;  
// count automatically promoted to float (5.0)
```

Explicit Casting (Programmer Forced)

- ▶ Performed **manually** by the programmer using the cast operator: ``(target_type) expression``.
- ▶ Used when demoting types or forcing floating-point division on integers.

```
int marks = 95, total = 200;  
float percentage;  
  
// Without cast: 95/200 = 0 (integer division)  
percentage = (float)marks / total * 100;  
// Result: 47.50%
```

Real-World Example: ATM Withdrawal

```
#include
#define SERVICE_FEE 1.50 // Constant macro

int main() {
    int withdrawAmount = 200; // Integer token
    double balance = 500.75; // Double token

    // Explicit cast & arithmetic expression
    double totalDeduction = (double)withdrawAmount + SERVICE_FEE;
    balance -= totalDeduction; // Shorthand operator

    printf("Withdrawn Amount: $%d\n", withdrawAmount);
    printf("Service Fee Applied: $%.2f\n", SERVICE_FEE);
    printf("Updated Balance: $%.2f\n", balance);
    return 0;
}
```

Code Breakdown & Concepts Used

- ▶ **Constants:** `SERVICE\_FEE` is fixed at `\$1.50` using `#define`.
- ▶ **Variables:** `balance` (double) and `withdrawAmount` (int) store user runtime data.
- ▶ **Explicit Cast:** `(double)withdrawAmount` converts integer to double before adding the decimal fee.
- ▶ **Compound Operator:** `balance -= totalDeduction` updates balance directly.
- ▶ **Formatted I/O:** `%d` formats integers, while `%.2f` formats doubles to 2 decimal places.

Unit 2 Summary & Exam Key Points

Pipeline

Remember the 4 build stages: **Preprocessor** (`.i`) → **Compiler** (`.s`) → **Assembler** (`.o`) → **Linker** (`.exe`).

Data & Memory

Know sizes: char (1B), int (2/4B), float (4B), double (8B). Remember 32 keywords cannot be variable names.

Operators

Practice operator precedence (Parentheses > Arithmetic > Relational > Logical > Assignment) and Type Casting formulas.